AI505

Optimization

Linear Constrained Optimization

Marco Chiarandini

Department of Mathematics & Computer Science
University of Southern Denmark

Outline

1. Linear Programming

2. Simplex Algorithm

3. Duality

4. Primal-Dual Hybrid Gradient (PDHG) Method for LP

Problem Formulation

- If an optimization problem has a linear objective and constraints, it is called a **linear programming problem (linear program, LP)**

- The general form of a linear program is:

$$
\begin{aligned}
\underset{\mathbf{x}}{\text{minimize}} \quad & \mathbf{c}^T\mathbf{x} \\
\text{subject to} \quad & A\mathbf{x} \leq \mathbf{b} \\
& D\mathbf{x} \geq \mathbf{e} \\
& F\mathbf{x} = \mathbf{g} \\
& \\
& \mathbf{x}, \mathbf{c} \in \mathbb{R}^n, \\
& A \in \mathbb{R}^{m \times n}, \mathbf{b} \in \mathbb{R}^m \\
& D \in \mathbb{R}^{p \times n}, \mathbf{e} \in \mathbb{R}^p \\
& F \in \mathbb{R}^{q \times n}, \mathbf{g} \in \mathbb{R}^q
\end{aligned}
$$

Numerical Example

$$
\begin{aligned}
\underset{x_1, x_2, x_3}{\text{minimize}} \quad & 2x_1 - 3x_2 + 7x_3 \\
\text{subject to} \quad & 2x_1 + 3x_2 - 8x_3 \leq 5 \\
& 4x_1 + x_2 + 3x_3 \leq 9 \\
& x_1 - 5x_2 - 3x_3 \geq -4 \\
& x_1 + x_2 + 2x_3 = 1
\end{aligned}
$$

Modelling in Linear Programming

Example

Given a set of assets/items I, each item with a price p_i per unit of measure and a value v_i for i in I, select the subset of items/assets that maximizes the total (expected return)value collected subject to a total expense that does not exceed a given budget B.

$$\begin{aligned} \max \quad & \sum_{i \in I} v_i x_i \\ \text{s.t.} \quad & \sum_{i \in I} p_i x_i \leq B \\ & x_i \geq 0, \quad \text{for all } i \text{ in } I \end{aligned}$$

This problem admits an efficient solution anyway.

Note: if indivisible items rather than assets are considered then the variables must be integer-valued and the problem becomes an **integer linear program** and is weakly NP-hard.

Modelling in Linear Programming

Many problems can be converted into linear programs that have the same solution.

Example

$$\text{minimize } L_1 = \|A\mathbf{x} - \mathbf{b}\|_1$$

$$\begin{aligned} \min\ & \mathbf{1}^T\mathbf{s} \\ \text{s.t. } & A\mathbf{x} - \mathbf{b} \leq \mathbf{s} \\ & -(A\mathbf{x} - \mathbf{b}) \leq \mathbf{s} \end{aligned}$$

Sum of absolute errors
Robust regression: Ignores outliers; focuses on the "median"fit.

Example

$$\text{minimize } L_\infty = \|A\mathbf{x} - \mathbf{b}\|_\infty$$

$$\begin{aligned} \min\ & t \\ \text{s.t. } & A\mathbf{x} - \mathbf{b} \leq t\mathbf{1} \\ & -(A\mathbf{x} - \mathbf{b}) \leq t\mathbf{1} \end{aligned}$$

Minimax Approximation or Chebyshev Approximation problem

Outline

1. Linear Programming

2. Simplex Algorithm

3. Duality

4. Primal-Dual Hybrid Gradient (PDHG) Method for LP

Problem Formulation

Every, general form, linear program can be rewritten more compactly in **standard form**

$$
\begin{aligned}
\underset{\mathbf{x}}{\text{minimize}} \quad & \mathbf{c}^T\mathbf{x} \\
\text{subject to} \quad & A\mathbf{x} \leq \mathbf{b} \\
& \mathbf{x} \geq \mathbf{0} \\
& \\
& \mathbf{x}, \mathbf{c} \in \mathbb{R}^n, \\
& A \in \mathbb{R}^{m \times n}, \mathbf{b} \in \mathbb{R}^m
\end{aligned}
$$

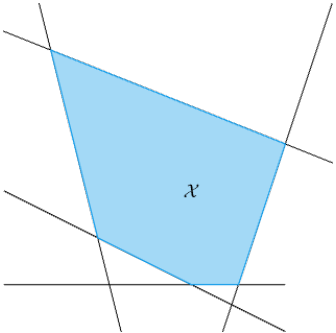
Example

$$
\begin{aligned}
\text{minimize } & 5x_1 + 4x_2 \\
\text{s.t. } & 2x_1 + 3x_2 \leq 5 \\
& 4x_1 + x_2 \leq 11
\end{aligned}
$$

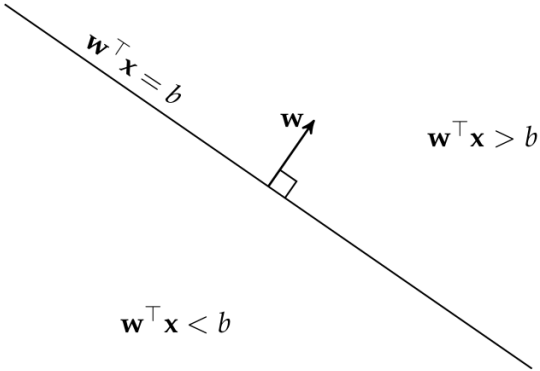
Problem Formulation

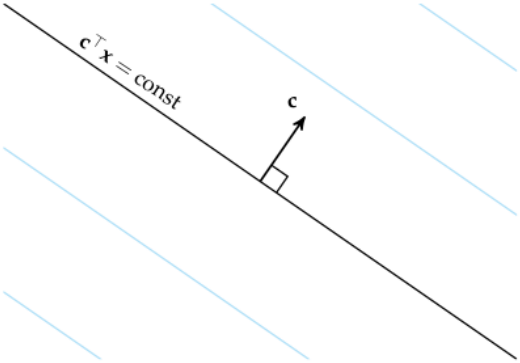
- Each inequality constraint defines a planar boundary of the feasible set called a **half-space**
- The set of inequality constraints define the intersection of multiple half-spaces forming a **convex set**
- Convexity of the feasible set, along with convexity of the objective function, implies that if we find a local feasible minimum, it is also a global feasible minimum.

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & \mathbf{c}^T\mathbf{x} \\ \text{subject to} \quad & A\mathbf{x} \leq \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned}$$



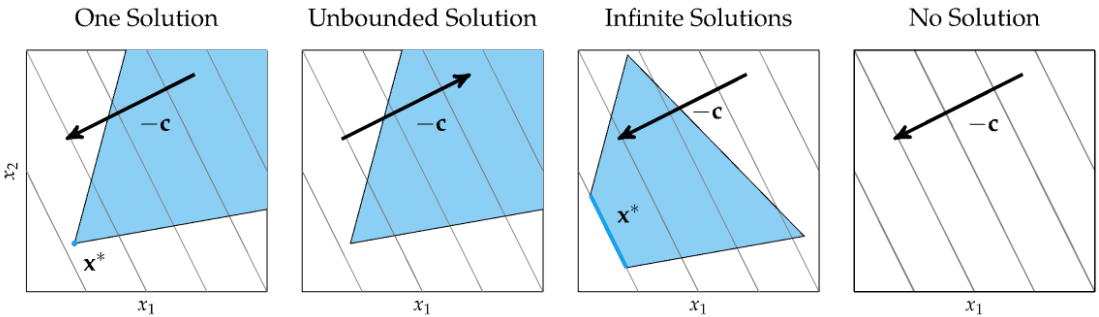
Half-Spaces and Supporting Hyperplanes





Problem Formulation

- How many solutions are there?



Problem Formulation

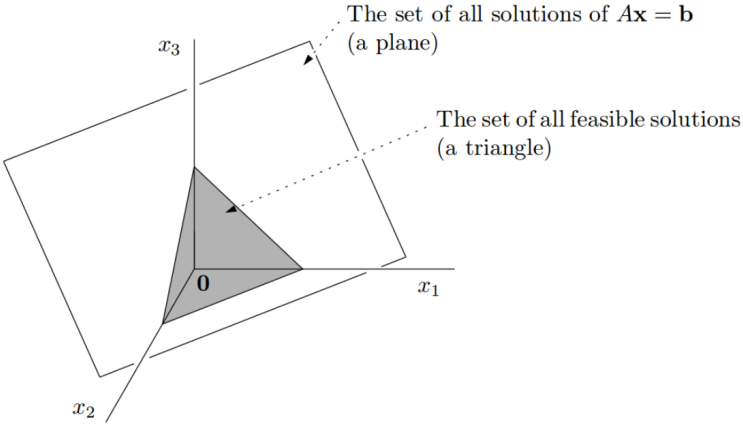
Linear programs are often solved in **equality form**

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & \mathbf{c}^T\mathbf{x} \\ \text{subject to} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned}$$

$\mathbf{x}, \mathbf{c} \in \mathbb{R}^{2n+m}$,

$A \in \mathbb{R}^{m \times 2n+m}$,

$\mathbf{b} \in \mathbb{R}^m$



Simplex Algorithm

- Guaranteed to solve any feasible and bounded linear program
- Works on the equality form
- Assumes that rows of A are linearly independent and $m \leq n'$ ($n' \leq 2n + m$)
- The feasible set of a linear program forms a **polytope** (polyhedron bounded by faces of $n - 1$ dimension)
- The simplex algorithm moves between **vertices** of the polytope until it finds an optimal **vertex**
- Points on faces not perpendicular to $\mathbf{c}$ can be improved by sliding along the face in the direction of the projection of $-\mathbf{c}$ onto the face.

Fundamental Theorem of LP

Theorem (Fundamental Theorem of Linear Programming)

Given:

$$\min\{\mathbf{c}^T\mathbf{x} \mid \mathbf{x} \in P\} \text{ where } P = \{\mathbf{x} \in \mathbb{R}^n \mid A\mathbf{x} \leq \mathbf{b}\}$$

If P is a bounded polyhedron and not empty and x^* is an optimal solution to the problem, then:

- $\mathbf{x}^*$ *is an extreme point (vertex) of P, or*
- $\mathbf{x}^*$ *lies on a face $F \subset P$ of optimal solution*

Proof:

- assume $\mathbf{x}^*$ not a vertex of P then $\exists$ a ball around it still in P. Show that a point in the ball has better cost
- if $\mathbf{x}^*$ is not a vertex then it is a convex combination of vertices. Show that all points are also optimal.

Simplex Algorithm

- Every vertex for a linear program in equality form can be uniquely defined by $n - m$ components of $\mathbf{x}$ that equal zero.
- choosing m design variables and setting the remaining variables to zero effectively removes $n - m$ columns of A, yielding an $m \times m$ constraint matrix
- the m selected columns of the matrix A are called **basis** and denoted by B: $x_i \geq 0$ for $i \in B$
- the $n - m$ columns not in B are called **not in basis** and are denoted by N: $x_i = 0$ for $i \in N$.

$$A\mathbf{x} = A_B\mathbf{x}_B = \mathbf{b} \quad \Longrightarrow \quad \mathbf{x}_B = A_B^{-1}\mathbf{b}$$

Simplex Algorithm

- every vertex has an associated partition (B, N),
- not every partition corresponds to a vertex.
  A_B might be not invertible or the point $\mathbf{x}_B$ might not be ≥ 0.
- identifying partitions that correspond to vertices corresponds to solving an LP problem as well!

Two phases of the algorithm

1. **Initialization Phase**: finding a feasible starting vertex
2. **Optimization Phase**: finding the optimal vertex

Simplex Algorithm: FONCs

Lagrangian function:

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\mu} \geq \mathbf{0}, \boldsymbol{\lambda}) = \mathbf{c}^T\mathbf{x} - \boldsymbol{\mu}^T\mathbf{x} - \boldsymbol{\lambda}^T(A\mathbf{x} - \mathbf{b})$$

The necessary conditions for optimality (KKT) are also sufficient for linear programs

- feasibility: $A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}$
- dual feasibility: $\boldsymbol{\mu} \geq \mathbf{0}$
- complementary slackness: $\boldsymbol{\mu} \cdot \mathbf{x} = \mathbf{0}$
- stationarity: $A^T\boldsymbol{\lambda} + \boldsymbol{\mu} = \mathbf{c}$

$$A^T \boldsymbol{\lambda} + \boldsymbol{\mu} = \mathbf{c} \quad \Longrightarrow \quad \begin{cases} A_B^T \boldsymbol{\lambda} + \boldsymbol{\mu}_B = \mathbf{c}_B \\ A_N^T \boldsymbol{\lambda} + \boldsymbol{\mu}_N = \mathbf{c}_N \end{cases}$$

- We can choose $\boldsymbol{\mu}_B = \mathbf{0}$ to satisfy complementary slackness (because $\mathbf{x}_B \geq 0$)

$$\boldsymbol{\mu}_N = \mathbf{c}_N - \left(A_B^{-1} A_N\right)^T \mathbf{c}_B$$

- Knowing $\boldsymbol{\mu}_N$ allows us to assess the optimality of the vertices. If $\boldsymbol{\mu}_B$ contains negative components, then dual feasibility is not satisfied and the vertex is sub-optimal.

- maintain a partition (B, N), which corresponds to a vertex of the feasible set polytope.

- The partition can be updated by swapping indices between B and N. Such a swap equates to moving from one vertex along an edge of the feasible set polytope to another vertex.

Simplex Algorithm: Optimization Phase

Pivoting

- $q \in V$ to enter in B

$$A\mathbf{x}' = A_B\mathbf{x}'_B + A_{\{q\}}x'_q = A_B\mathbf{x}_B = A\mathbf{x} = \mathbf{b}$$

- $p \in B$ to leave B becomes zero during the transition.

$$\mathbf{x}'_B = \mathbf{x}_B - A_B^{-1}A_{\{q\}}x'_q \qquad \Longrightarrow \qquad (\mathbf{x}'_B)_p = 0 = (\mathbf{x}_B)_p - \left(A_B^{-1}A_{\{q\}}\right)_p x'_q$$

- leaving index is obtained using the **minimum ratio test**: compute x'_q for each potential leaving index p and select the leaving index p that yields the smallest x'_q.

- Choosing an entering index q decreases the objective function value by

$$\mathbf{c}^T\mathbf{x}' = \mathbf{c}_B^T\mathbf{x}'_B + c_q x'_q = \mathbf{c}^T\mathbf{x} + \mu_q x'_q$$

- The objective function decreases only if μ_q is negative.

Simplex Algorithm: Optimization Phase

- In order to progress toward optimality, we must choose an index q in N such that μ_q is negative. If all components of $\boldsymbol{\mu}_N$ are non-negative, we have found a global optimum.

- Since there can be multiple negative entries in $\boldsymbol{\mu}_N$, several possible heuristics to search for optimal vertex (choose next q)

  - **Dantzig's rule**: choose most negative entry in $\boldsymbol{\mu}_N$; easy to calculate
  - **Greedy heuristic (largest decrease)**: maximally reduces objective at each step
  - **Bland's rule**: chooses first vertex found with negative $\boldsymbol{\mu}_N$; useful for preventing or breaking out of cycles

Simplex Algorithm: Initialization Phase

- The starting vertex of the optimization phase is found by solving an additional **auxiliary linear program** that has a known feasible starting vertex

$$\underset{\mathbf{x},\mathbf{z}}{\text{minimize}} \quad \begin{bmatrix} \mathbf{0}^T & \mathbf{1}^T \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \mathbf{z} \end{bmatrix}$$

$$\begin{bmatrix} A & Z \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \mathbf{z} \end{bmatrix} = \mathbf{b}$$

$$\begin{bmatrix} \mathbf{x} \\ \mathbf{z} \end{bmatrix} \geq \mathbf{0}$$

- The solution is a feasible vertex in the original linear program

Outline

1. Linear Programming

2. Simplex Algorithm

3. Duality

4. Primal-Dual Hybrid Gradient (PDHG) Method for LP

Dual Certificates

- Verification that the solution returned by the algorithm is actually the correct solution
- Recall that the solution to the dual problem, d^* provides a lower bound to the solution of the primal problem, p^*
- If $d^* = p^*$ then p^* is guaranteed to be the unique optimal value because the duality gap is zero
- What happens if one of the two is unbounded or infeasible?

Dual Certificates

Linear programs have a simple dual form:

Primal form (equality)

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & \mathbf{c}^T\mathbf{x} \\ \text{subject to} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned}$$

Dual form

$$\begin{aligned} \underset{\lambda}{\text{maximize}} \quad & \mathbf{b}^T\boldsymbol{\lambda} \\ \text{subject to} \quad & A^T\boldsymbol{\lambda} \leq \mathbf{c} \end{aligned}$$

Strong Duality Theorem

Due to Von Neumann and Dantzig 1947 and Gale, Kuhn and Tucker 1951.

Theorem (Strong Duality Theorem)

Given:

$$(P) \qquad \min\{\mathbf{c}^T\mathbf{x} \mid A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$$

$$(D) \qquad \max\{\mathbf{b}^T\boldsymbol{\lambda} \mid A^T\boldsymbol{\lambda} \geq \mathbf{c}\}$$

exactly one of the following occurs:

1. *(P) and (D) are both infeasible*
2. *(P) is unbounded and (D) is infeasible*
3. *(P) is infeasible and (D) is unbounded*
4. *(P) has feasible solution, then let an optimal be:* $\mathbf{x}^* = [x_1^*, \ldots, x_n^*]$
   (D) has feasible solution, then let an optimal be: $\boldsymbol{\lambda}^* = [\lambda_1^*, \ldots, \lambda_m^*]$*, then:*

$$p^* = \mathbf{c}^T\mathbf{x}^* = \mathbf{b}^T\boldsymbol{\lambda}^* = d^*$$

Summary

- Linear programs are problems consisting of a linear objective function and linear constraints
- The simplex algorithm can optimize linear programs globally in an efficient manner
- Dual certificates allow us to verify that a candidate primal-dual solution pair is optimal
- Linear programs can be solved to optimality for problems with millions of variables.

Outline

1. Linear Programming

2. Simplex Algorithm

3. Duality

4. Primal-Dual Hybrid Gradient (PDHG) Method for LP

First Order Methods

FOM requires only matrix-vector multiplication:

1. Matrix-vector multiplication is very suitable for utilizing modern hardware and distributed computation
2. Matrix factorization free. The memory usage is relatively low.

FOM is particularly good at solving large instances

With help of GPU, recent research shows PDHG is fast when the problem becomes large, see:
["cuPDLP. jl: A GPU implementation of restarted primal-dual hybrid gradient for linear programming in Julia", Lu/Yang, arXiv:2311.12180, 2023.]
["cuPDLP-C: A Strengthened Implementation of cuPDLP for Linear Programming by C language," Lu et al. arXiv:2312.14832, 2023.]

Problem Formulation

For the standard LP:

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & \mathbf{c}^T\mathbf{x} \\ \text{subject to} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned}$$

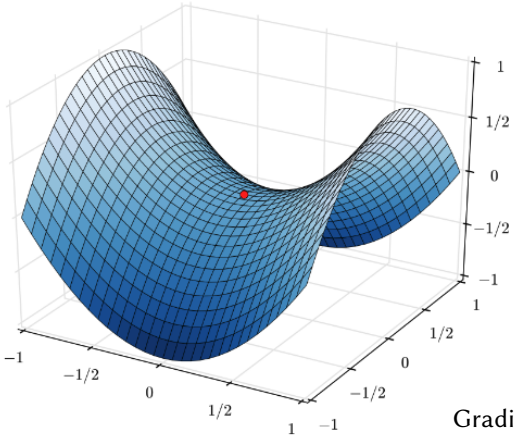
Via Lagrangian function:

$$\min_{\mathbf{x}\geq\mathbf{0}} \max_{\mathbf{y}} \mathcal{L}(\mathbf{x}, \mathbf{y}) \stackrel{def}{=} \mathbf{c}^T\mathbf{x} - \mathbf{y}^T(A\mathbf{x} - \mathbf{b})$$

Saddle point: The $(\mathbf{x}^*, \mathbf{y}^*)$ with $\mathbf{x}^* \geq \mathbf{0}$ is a saddle point of $\mathcal{L}(\mathbf{x}, \mathbf{y})$ if for any $\mathbf{x} \geq \mathbf{0}$ and $\mathbf{y}$:

$$\mathcal{L}(\mathbf{x}^*, \mathbf{y}) \leq \mathcal{L}(\mathbf{x}^*, \mathbf{y}^*) \leq \mathcal{L}(\mathbf{x}, \mathbf{y}^*)$$

Gradient Descent Ascent



Gradient Descent Ascent:

$$\mathbf{x}^{k+1} = \text{Projection}_{\mathbf{x}\geq\mathbf{0}}\left(\mathbf{x}^k - \alpha(\mathbf{c} - A^T\mathbf{y}^k)\right) = \left(\mathbf{x}^k - \alpha(\mathbf{c} - A^T\mathbf{y}^k)\right)^+$$

$$\mathbf{y}^{k+1} = \mathbf{y}^k + \alpha(\mathbf{b} - A\mathbf{x}^{k+1})$$

Gradient Descent Ascent may not converge to the saddle point
Example:

$$\max_{\mathbf{x}} \min_{\mathbf{y}} \mathbf{x}\mathbf{y}$$

we have closed form of each iteration:

$$\begin{bmatrix} \mathbf{x}^k \\ \mathbf{y}^k \end{bmatrix} = \begin{bmatrix} 1 & \alpha \\ -\alpha & 1 \end{bmatrix}^k \begin{bmatrix} \mathbf{x}^0 \\ \mathbf{y}^0 \end{bmatrix}$$

The determinant of $\left[\begin{smallmatrix} 1 & \alpha \\ -\alpha & 1 \end{smallmatrix}\right]$ is $1 + \alpha^2 > 1$, no matter how small α is, which means this approach never converges.

Primal-Dual Hybrid Gradient (PDHG) Method

PDHG: To update $\mathbf{y}^{k+1}$ uses the idea of momentum ["A Primal-Dual Hybrid Gradient Method for Convex Problems with Applications to Imaging," Chambolle and Pock, Journal of Mathematical Imaging and Vision, 2011.]:

$$\mathbf{x}^{k+1} = \text{Projection}_{\mathbf{x}\geq\mathbf{0}}\left(\mathbf{x}^k - \alpha(\mathbf{c} - A^T\mathbf{y}^k)\right) = \left(\mathbf{x}^k - \alpha(\mathbf{c} - A^T\mathbf{y}^k)\right)^+$$
$$\mathbf{y}^{k+1} = \mathbf{y}^k + \alpha\left(\mathbf{b} - A(2\mathbf{x}^{k+1} - \mathbf{x}^k)\right)$$

Example:

$$\max_{\mathbf{x}} \min_{\mathbf{y}} \mathbf{x}\mathbf{y}$$

we still have closed form of each iteration:

$$\begin{bmatrix} \mathbf{x}^k \\ \mathbf{y}^k \end{bmatrix} = \begin{bmatrix} 1 & \alpha \\ -\alpha & 1-2\alpha^2 \end{bmatrix}^k \begin{bmatrix} \mathbf{x}^0 \\ \mathbf{y}^0 \end{bmatrix}$$

The modulus of eigenvalues of $\left[\begin{smallmatrix} 1 & \alpha \\ -\alpha & 1-2\alpha^2 \end{smallmatrix}\right]$ are smaller than 1 if $\alpha < 1$ so PDHG converges.